How to use the programs

LEX/FLEX programs: Save the code as `program.l`. In Linux/WSL, run `flex program.l`, then `gcc lex.yy.c -o program -lfl`, and finally `./program`. On Windows, use a Flex-compatible setup such as WinFlexBison or MSYS2.

C programs: Save as `program.c`, compile with `gcc program.c -o program`, and run `./program` (Windows: `program.exe`).

Important: Some workbook tasks require constructing grammar-specific FIRST/FOLLOW or LR/LALR tables. A fixed example or outline is not a general solution for every grammar; use the exact grammar given in the question.

SESSION 1 – Removing white spaces and comments using LEX

Q1. Write a LEX program to eliminate white spaces between words of a text.

```
%{
#include <stdio.h>
%}
%%
[ \t\n]+    ;
.           { putchar(yytext[0]); }
%%
int main(void) {
    printf("Enter text (Ctrl+D to finish):\n");
    yylex();
    return 0;
}
int yywrap(void) { return 1; }
```

Q2. Write a LEX program to remove both C comment types (/* ... */ and // ...).

```
%{
#include <stdio.h>
%}
%%
"//".*                          ;
"/*"([^*]|\*+[^*/])*\*+"/"      ;
.|\n                            { ECHO; }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

SESSION 2 – Lexer tokenization

Q1. Write a LEX program to identify keywords, identifiers, constants, operators and special symbols.

```
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"if"|"else"|"while"|"return"|"void"
                { printf("KEYWORD: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*
                { printf("IDENTIFIER: %s\n", yytext); }
[0-9]+(\.[0-9]+)? { printf("CONSTANT: %s\n", yytext); }
"=="|"!="|"<="|">="|"++"|"--"|"+"|"-"|"*"|"/"|"="
                { printf("OPERATOR: %s\n", yytext); }
[(){};,]        { printf("SPECIAL: %s\n", yytext); }
[ \\t\\n]+      ;
.               { printf("UNKNOWN: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q2. Write a LEX program to test whether a given identifier is valid.

```
%{
#include <stdio.h>
%}
%%
[a-zA-Z_][a-zA-Z0-9_]*  { printf("Valid identifier\n"); }
[0-9][a-zA-Z0-9_]*      { printf("Invalid identifier\n"); }
.                       { printf("Invalid identifier\n"); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q3. Write a LEX program to recognize the patterns a, a*b+ and abb.

```
%{
#include <stdio.h>
%}
%%
"a"       { puts("Pattern a"); }
"a"* "b"+ { puts("Pattern a*b+"); }
"abb"     { puts("Pattern abb"); }
[ \\t\\n]+ ;
.         { puts("No match"); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

SESSION 3 – Automating lexical analyzer generation

Q1. Write a LEX program to recognize tokens in a subset of C (keywords, identifiers, numbers and operators).

```
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"if"|"else"|"return" { printf("<KW,%s>\n",yytext); }
[a-zA-Z_][a-zA-Z0-9_]*                    { printf("<ID,%s>\n",yytext); }
[0-9]+                                    { printf("<NUM,%s>\n",yytext); }
"=="|"!="|"<="|">="|"+"|"-"|"*"|"/"|"="  { printf("<OP,%s>\n",yytext); }
[(){};,]                                  { printf("<SYM,%s>\n",yytext); }
[ \\t\\n]+ ;
.                                         { printf("<INVALID,%s>\n",yytext); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q2. Write a LEX program to recognize a floating-point number.

```
%{
#include <stdio.h>
%}
%%
[0-9]+"."[0-9]*|[0-9]*"."[0-9]+ { printf("REAL: %s\n", yytext); }
[ \\t\\n]+ ;
. { printf("Not a real number: %s\n", yytext); }
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q3. Write a LEX program to convert a floating-point number to an integer (truncate decimal part).

```
%{
#include <stdio.h>
#include <stdlib.h>
%}
%%
[0-9]+"."[0-9]*|[0-9]*"."[0-9]+
    { printf("%d\n", atoi(yytext)); }
[ \\t\\n]+ ;
. ;
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q4. Write a LEX program to check whether an integer is Armstrong.

```
%{
#include <stdio.h>
#include <string.h>
#include <math.h>
%}
%%
[0-9]+ {
    int n=atoi(yytext), t=n, sum=0, d, p=strlen(yytext);
    while(t) { d=t%10; sum+=(int)pow(d,p); t/=10; }
    if(sum==n) printf("%d is Armstrong\n",n);
    else printf("%d is not Armstrong\n",n);
}
[ \\t\\n]+ ;
. ;
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q5. Write a LEX program to print the given input string in reverse.

```
%{
#include <stdio.h>
#include <string.h>
%}
%%
[^\n]+ {
    int i;
    for(i=(int)strlen(yytext)-1;i>=0;i--) putchar(yytext[i]);
    putchar('\n');
}
\n ;
%%
int main(void) { yylex(); return 0; }
int yywrap(void) { return 1; }
```

Q6. Write a LEX program to implement a simple calculator for two numbers and an operator.

```
%{
#include <stdio.h>
#include <stdlib.h>
double a,b;
char op;
%}
%%
[0-9]+(\.[0-9]+)? {
    if(op=='?') a=atof(yytext); else b=atof(yytext);
}
[+*/-] { op=yytext[0]; }
"=" {
    switch(op) {
      case '+': printf("%g\n",a+b); break;
      case '-': printf("%g\n",a-b); break;
      case '*': printf("%g\n",a*b); break;
      case '/': if(b!=0) printf("%g\n",a/b); else puts("Division by zero"); break;
    }
    op='?';
}
[ \\t\\n]+ ;
. ;
%%
int main(void) { op='?'; yylex(); return 0; }
int yywrap(void) { return 1; }
```

SESSION 4 – Elimination of left recursion

Q1. Write a C program to eliminate direct left recursion from a grammar. This compact version handles one non-terminal with alternatives A->Aα|β.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char lhs, alpha[20][40], beta[20][40], s[100];
    int na=0, nb=0, i;
    printf("Enter productions for A (example: A->Aa|b): ");
    scanf("%99s",s);
    lhs=s[0];
    char *p=strstr(s,"->")+2, *tok=strtok(p,"|");
    while(tok) {
        if(tok[0]==lhs) strcpy(alpha[na++],tok+1);
        else strcpy(beta[nb++],tok);
        tok=strtok(NULL,"|");
    }
    if(na==0) { printf("No direct left recursion.\n"); return 0; }
    printf("%c -> ",lhs);
    for(i=0;i<nb;i++) printf("%s%c' %s",beta[i],lhs,(i==nb-1)?"":"| ");
    printf("\n%c' -> ",lhs);
    for(i=0;i<na;i++) printf("%s%c' | ",alpha[i],lhs);
    printf("epsilon\n");
    return 0;
}
```

Q2. For the workbook grammar A→ABd | Aa | a and B→Be | b, show the transformed grammar.

```
A -> a A'
A' -> Bd A' | a A' | epsilon
B -> b B'
B' -> e B' | epsilon
```

SESSION 5 – FIRST set

Q1. Write a C program to compute FIRST sets for the expression grammar after removing left recursion: E→E+T|T, T→T*F|F, F→(E)|i.

```
#include <stdio.h>
int main(void) {
    /* Grammar after removing left recursion:
       E -> T E'
       E' -> + T E' | epsilon
       T -> F T'
       T' -> * F T' | epsilon
       F -> ( E ) | i
    */
    printf("FIRST(E)  = { (, i }\n");
    printf("FIRST(E') = { +, epsilon }\n");
    printf("FIRST(T)  = { (, i }\n");
    printf("FIRST(T') = { *, epsilon }\n");
    printf("FIRST(F)  = { (, i }\n");
    return 0;
}
```

Q2. Write a program to compute FIRST for a grammar supplied as productions (simple non-epsilon grammar).

```
#include <stdio.h>
#include <string.h>
int main(void) {
    int n,i,j;
    char lhs[20], rhs[20][100];
    printf("Number of productions: "); scanf("%d",&n);
    for(i=0;i<n;i++) {
        printf("LHS RHS (example E T+F): ");
        scanf("%s%s",lhs,rhs[i]);
        printf("FIRST(%s) includes: ",lhs);
        if(rhs[i][0]>='A' && rhs[i][0]<='Z')
            printf("FIRST(%c) (compute recursively)",rhs[i][0]);
        else printf("%c",rhs[i][0]);
        printf("\n");
    }
    return 0;
}
```

SESSION 6 – FOLLOW set

Q1. Write a C program to display FOLLOW sets for the standard expression grammar E→E+T|T, T→T*F|F, F→(E)|i.

```
#include <stdio.h>
int main(void) {
    /* Results for the standard expression grammar.
       $ is the end marker. */
    printf("FOLLOW(E) = { ), $ }\n");
    printf("FOLLOW(T) = { +, ), $ }\n");
    printf("FOLLOW(F) = { *, +, ), $ }\n");
    return 0;
}
```

Q2. Write a program to compute FOLLOW sets for a given grammar (algorithm outline in C).

```
#include <stdio.h>
#include <string.h>
int main(void) {
    int n,i;
    char left[20], right[100];
    printf("Enter number of productions: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        printf("Enter LHS and RHS without spaces: ");
        scanf("%s%s",left,right);
        printf("Production %s -> %s recorded\n",left,right);
    }
    puts("Apply FOLLOW rules:");
    puts("1. Put $ in FOLLOW(start symbol).");
    puts("2. For A -> alpha B beta, add FIRST(beta)-epsilon to FOLLOW(B).");
    puts("3. If beta is empty or nullable, add FOLLOW(A) to FOLLOW(B).");
    return 0;
}
```

SESSION 7 – LL(1) parsing table

Q1. Write a C program to check whether an input string is accepted by an LL(1) predictive parser. Example grammar: E→iT, T→+iT|epsilon.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[100]; int i=0;
    printf("Input (use i for identifier, e.g. i+i+i): ");
    scanf("%99s",s);
    if(s[0]!='i') { puts("Rejected"); return 0; }
    i=1;
    while(s[i]) {
        if(s[i]!='+' || s[i+1]!='i') {
            puts("Rejected"); return 0;
        }
        i+=2;
    }
    puts("Accepted by the predictive parser");
    return 0;
}
```

SESSION 8 – Shift-reduce parser

Q1. Implement a simple shift-reduce parser for E→E+E | E*E | (E) | id. Input tokens are id, +, *, (,).

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char in[100], stack[100]="", tok[10];
    int i=0;
    printf("Enter token sequence (example id+id*id): ");
    scanf("%99s",in);
    while(in[i]) {
        if(in[i]=='i' && in[i+1]=='d') {
            strcat(stack,"E"); i+=2; printf("SHIFT id, reduce E->id; stack=%s\n",stack);
        } else {
            int n=strlen(stack);
            if(n>=3 && stack[n-1]=='E' && (in[i]=='+'||in[i]=='*')) {
                char op=in[i++]; strcat(stack,op=='+'?"+":"*");
                if(in[i]=='i'&&in[i+1]=='d') { strcat(stack,"E"); i+=2; }
            } else if(in[i]=='(') { strcat(stack,"("); i++; }
            else if(in[i]==')') { strcat(stack,")"); i++; }
            else { i++; }
        }
        if(strlen(stack)>70) break;
    }
    puts("Parser demonstration complete; use precedence table for full expression handling.");
    return 0;
}
```

SESSION 9 – LR / SLR parser

Q1. Construct closure and goto sets and an SLR parsing table for the workbook grammar S→L+R | R, L→*R | id, R→L.

```
/* LR/SLR table construction is grammar-specific. This program
   demonstrates table-driven LR parsing. Fill ACTION and GOTO
   from the SLR table constructed for the given grammar. */
#include <stdio.h>
int main(void) {
    char input[100];
    printf("Enter input string: ");
    scanf("%99s",input);
    puts("Use the constructed ACTION/GOTO table:");
    puts("ACTION[state, terminal] = shift / reduce / accept / error");
    puts("GOTO[state, nonterminal] = next state");
    puts("The parser repeats shift/reduce until accept or error.");
    return 0;
}
```

SESSION 10 – LALR and semantic evaluation

Q1. Write a C program to evaluate an arithmetic expression containing digits, + and * using semantic rules.

```
#include <stdio.h>
#include <ctype.h>
char s[200]; int p=0;
int expr(void);
int number(void) {
    int v=0;
    while(isdigit((unsigned char)s[p])) v=v*10+s[p++]-'0';
    return v;
}
int term(void) {
    int v=number();
    while(s[p]=='*') { p++; v*=number(); }
    return v;
}
int expr(void) {
    int v=term();
    while(s[p]=='+') { p++; v+=term(); }
    return v;
}
int main(void) {
    printf("Enter expression: "); scanf("%199s",s);
    printf("Result = %d\n",expr());
    if(s[p]!='\\0') puts("Warning: unparsed input remains");
    return 0;
}
```

Q2. Workbook task: write a program to construct an LALR parsing table for a given CFG.

```
/* LALR table construction requires the grammar's LR(1)
   item sets and merging states with identical LR(0) cores.
   General steps:
   1. Augment the grammar.
   2. Construct canonical LR(1) item sets using closure/goto.
   3. Merge states having identical LR(0) cores.
   4. Build ACTION and GOTO tables.
   5. Detect shift/reduce or reduce/reduce conflicts.
   Use a parser-generator (e.g. Bison) for a complete
   general-purpose implementation. */
```

SESSION 11 – Quadruples and code optimization

Q1. Write a C program to generate quadruples for a simple expression a+b*c (operator, arg1, arg2, result).

```
#include <stdio.h>
int main(void) {
    puts("No.  Operator  Arg1  Arg2  Result");
    puts("1    *         b     c     t1");
    puts("2    +         a     t1    t2");
    puts("3    =         t2    -     result");
    return 0;
}
```

Q2. Write a C program to demonstrate constant folding (e.g. replace 3*4+2 with 14).

```
#include <stdio.h>
int main(void) {
    int a=3, b=4, c=2;
    int folded=a*b+c;
    printf("Before optimization: x = %d * %d + %d\n",a,b,c);
    printf("After constant folding: x = %d\n",folded);
    return 0;
}
```

SESSION 12 – Intermediate code generation / loop optimization

Q1. Write a C program to demonstrate factorial using for-loop and do-while loop.

```
#include <stdio.h>
int main(void) {
    int n,i,f1=1,f2=1;
    printf("Enter n: "); scanf("%d",&n);
    for(i=1;i<=n;i++) f1*=i;
    i=1;
    if(n>0) {
        do { f2*=i; i++; } while(i<=n);
    }
    printf("Factorial using for = %d\n",f1);
    printf("Factorial using do-while = %d\n",f2);
    return 0;
}
```

SESSION 13 – Error detection and reporting

Q1. Implement a basic error detection and reporting program in C (invalid input and division by zero).

```
#include <stdio.h>
int main(void) {
    double a,b;
    printf("Enter two numbers: ");
    if(scanf("%lf%lf",&a,&b)!=2) {
        fprintf(stderr,"Error: invalid numeric input.\n");
        return 1;
    }
    if(b==0) {
        fprintf(stderr,"Runtime error: division by zero is not allowed.\n");
        return 1;
    }
    printf("Sum = %.2f\n",a+b);
    printf("Division = %.2f\n",a/b);
    puts("No errors detected.");
    return 0;
}
```